



Spam · Ham · Smishing Detector

Technical Documentation

NLP Project · Spring 2025–2026

Supervisor	Dr. Wafaa Samy
Language	Python 3.11+
Framework	scikit-learn · tkinter

Team Members

Ali Yahia · Eslam Mahmoud · Yassen Abo Alhassan
Mohammed Alaa · Ahmed Emad

1. Project Overview

The Spam · Ham · Smishing Detector is a machine-learning-powered desktop application that classifies SMS messages into three distinct categories. Six classification algorithms are trained on a 10,191-row balanced dataset and exposed through a real-time GUI built with Python's tkinter library. Every run produces a majority-vote verdict so the system degrades gracefully even when individual models disagree.

1.1 Message Classes

The classifier distinguishes three categories of SMS message. Understanding the boundary between them — especially between SPAM and SMISHING — is critical to interpreting results correctly.

Class	Definition	Typical Examples
HAM	Normal personal messages between real people — no commercial or fraudulent intent.	<i>"Hey bro, are you free tonight?", "Congrats on passing the exam!", "Call me when you're done."</i>
SPAM	Commercial SMS — unsolicited marketing from businesses. Covers ringtones, mobile subscriptions, adult chat lines, and retail/shopping offers.	<i>"Free Nokia ringtone! Txt NOKIA to 8007", "Subscribe to mobile content for £3/week", "Work from home earn unlimited income daily"</i>
SMISHING	Fraudulent SMS designed to deceive recipients. Covers both prize/lottery fraud (fake winnings) AND bank/account fraud (phishing for credentials or personal info).	<i>"You won £5000! Call NOW", "Your Barclays account is locked — verify at hsbc-secure.com"</i>

The key distinction between SPAM and SMISHING is intent and mechanism: SPAM monetises through a commercial transaction (buy a ringtone, subscribe to a service), while SMISHING monetises through deception (steal prize-claim fees, harvest bank credentials). Both may promise financial rewards, but SMISHING always uses fraudulent framing and typically provides a phone number to call or a phishing URL.

1.2 Key Design Decisions

- Single-file architecture: training pipeline and GUI are co-located in `spam_detector.py` to prevent sklearn version mismatches between a separately trained model and the GUI runtime.
- Models retrain on first run using the local sklearn version, then save to `saved_models_final/`. Subsequent runs load the pickled files instantly.
- 53 hard-coded training samples cover the trickiest boundary cases — messages that look superficially like one class but belong to another.
- Majority vote across all six models is displayed as the VERDICT, providing robustness against any individual model's failure mode.

2. Dataset

2.1 Overview

The project uses "A Balanced Dataset for Spam and Smishing Detection" (Dataset_10191.csv). The dataset is perfectly balanced across all three classes, which eliminates class-imbalance bias from the evaluation.

Class	Count	Share	Notes
HAM	3,397	33.3%	Personal conversations, casual messages, real-life communication
SPAM	3,397	33.3%	Commercial SMS (ringtones, subscriptions, adult, retail)
SMISHING	3,397	33.3%	Fraudulent SMS (prize/lottery fraud + bank/account fraud)
TOTAL	10,191	100%	Perfectly balanced — no class weighting needed for baseline

2.2 Dataset Columns

- TEXT — the raw SMS message content (primary input).
- LABEL — ground-truth class: ham, spam, or smishing.
- URL — binary flag indicating whether a URL is present in the message.
- EMAIL — binary flag indicating whether an email address is present.
- PHONE — binary flag indicating whether a phone number is present.

2.3 Hard Training Samples

53 hand-crafted samples are concatenated to the dataset before training. These target boundary cases identified during development. They cover four sub-groups:

- HAM that looks like spam/smishing — casual messages containing words like "free", "won", "cash", "bonus" in a personal context (e.g., "Congrats bro you passed the exam!").
- True SPAM (commercial) — ringtone and subscription SMS, adult chat-line solicitations, work-from-home scam ads.
- True SMISHING (prize/lottery) — messages with prize amounts AND a phone number to call.
- True SMISHING (bank/account) — messages impersonating banks or platforms with phishing URLs.

3. Feature Engineering

3.1 Text Preprocessing Pipeline

All text passes through a standardised preprocess() function before vectorisation:

- Lowercasing.
- URL replacement with the token urllink.
- Email address replacement with emailaddr.
- Phone number replacement with phonenumber (regex targets 7+ digit sequences).
- Punctuation removal.
- Digit replacement with numbr.
- Lightweight suffix stripping (stem()) for -tion, -ing, -ment, -ness, -ful, -less, -ous, -ive, -ed, -ly, -er, -al, and plural -s/-es/-ies.
- Stopword removal using a 100-term English stopwords list.

3.2 TF-IDF Vectorisation

A TfidfVectorizer is fitted on the preprocessed corpus with the following settings:

- n-gram range: unigrams through trigrams (1, 3).
- Maximum vocabulary size: 6,000 features.
- Sublinear TF scaling: enabled ($\log(1 + \text{tf})$) to reduce the influence of very frequent terms.
- Minimum document frequency: 1 (all observed n-grams are kept).

3.3 Hand-Crafted Features (25 total)

A parallel feature vector of 25 hand-crafted signals is extracted from the raw (not preprocessed) text. These capture semantic patterns that TF-IDF misses. The hand features are scaled with MaxAbsScaler so they are not dwarfed by the high-dimensional TF-IDF matrix, then horizontally stacked before training.

Feature Name	Type	Description
has_url	Binary	Presence of a URL or short link
has_email	Binary	Presence of an email address
has_phone	Binary	Phone number matching UK/India patterns
has_shortcode	Binary	SMS shortcode pattern (txt WORD to 5-digit number)
has_bank_brand	Binary	Presence of bank/platform brand names (PayPal, HSBC, Apple...)
has_spam_comm	Binary	Hit on commercial spam keyword list
has_smish_prize	Binary	Hit on prize/lottery smishing keyword list
has_smish_bank	Binary	Hit on bank/account fraud keyword list
has_ham	Binary	Hit on casual personal-conversation keyword list
prize_and_phone	Interaction	Prize keyword AND phone number → strong smishing signal
bank_and_contact	Interaction	Bank brand AND (URL or phone) → bank-fraud smishing
comm_and_code	Interaction	Commercial keyword AND shortcode → classic spam
prize_no_phone	Interaction	Prize keyword WITHOUT phone → more spam-like

Feature Name	Type	Description
pure_spam_comm	Interaction	Commercial keyword WITHOUT prize terms
pure_ham	Interaction	Ham keyword with no suspicious signals
urgency	Style (0–3)	Count of urgency phrases ("urgent", "act now", "expires soon"…)
caps_ratio	Style (0–1)	Fraction of uppercase characters
excl_count	Style (0–5)	Number of exclamation marks (capped at 5)
word_count	Style	Total number of words in the message
has_gbp	Style	Presence of the £ symbol
has_rupee	Style	Presence of Rs./INR/₹ symbols
has_shorturl	Style	Short URL service (bit.ly, tinyurl, goo.gl, t.co)
link_urgency	Interaction	URL × urgency score
phone_urgency	Interaction	Phone × urgency score
casual_tone	Style	Casual language markers (bro, dude, lol, omg, btw…)

3.4 Combined Feature Matrix

The final feature matrix fed to most models is the horizontal concatenation of:

- 6,000-dimensional TF-IDF sparse matrix.
- 25 scaled hand-crafted features (dense → converted to sparse before stacking).

Naive Bayes is the exception: it requires non-negative inputs, so it is trained and evaluated on the TF-IDF matrix only.

4. Classification Models

4.1 Algorithm Summary

Algorithm	Feature Input	Key Strength
Logistic Regression	TF-IDF + hand-crafted	Fast, interpretable, strong baseline
Random Forest	TF-IDF + hand-crafted	Robust, handles feature interactions
SVM (LinearSVC + Calibration)	TF-IDF + hand-crafted	High-margin text classifier
Naive Bayes (MultinomialNB)	TF-IDF only	Excellent for term-frequency patterns
Decision Tree	TF-IDF + hand-crafted	Interpretable rule-based splitting
K-Nearest Neighbors	TF-IDF + hand-crafted (cosine)	Similarity-based, no assumptions

4.2 Training Configuration

All models use `class_weight='balanced'` (or equivalent) to counteract residual class imbalance in the augmented dataset after hard samples are added. The train/test split is 80/20 with stratification.

- Logistic Regression: `C=5`, `max_iter=2000`, `solver=lbfgs`, `multi_class=auto`.
- Random Forest: 300 estimators, unlimited depth, `n_jobs=-1`.
- SVM: `LinearSVC(C=1.0, max_iter=3000)` wrapped in `CalibratedClassifierCV` for probability output.
- Naive Bayes: `MultinomialNB(alpha=0.1)` — trained on TF-IDF only.
- Decision Tree: `max_depth=20`.
- KNN: `k=7`, cosine metric, `n_jobs=-1`.

4.3 Prediction & Majority Vote

At inference time, each model produces a predicted class label and a probability distribution over the three classes. The GUI displays per-model results in a table showing confidence and the full probability breakdown (HAM %, SPAM %, SMISHING %).

The VERDICT banner shows the majority-vote winner (the most frequently predicted class across all models) and the agreement count (e.g., "4/6 models agree"). In case of a tie, the first-sorted label wins.

5. Classification Decision Logic

5.1 Keyword Groups

Four disjoint keyword lists drive the hand-crafted binary features. Each list targets a specific class boundary:

- SPAM commercial (`_SPAM_COMMERCIAL`): ringtones, subscriptions, adult chat, retail offers, work-from-home income claims, "not a scam" denials.
- SMISHING prize (`_SMISHING_PRIZE`): "you have won", "winner", "prize guaranteed", currency amounts (£500–£10,000, Rs.2,00,000), holiday/travel keywords.
- SMISHING bank (`_SMISHING_BANK`): "suspended", "verify", "unusual activity", bank/platform brand triggers, "action required", KYC, "compromised".
- HAM (`_HAM_KW`): casual language (bro, dude, lol), social activities (lunch, party, hang out), school context, emotional messages.

5.2 Critical Boundaries

The hardest classification boundary is SPAM vs. SMISHING when both mention money or prizes. The system resolves this through interaction features:

- Prize keyword + phone number → SMISHING (prize fraud always provides a number to call).
- Prize keyword without phone number → leans SPAM (commercial lottery-style promotion).

- Bank brand + URL or phone → SMISHING (bank impersonation phishing).
- Commercial keyword + SMS shortcode → SPAM (paid subscription model).

5.3 Tricky Case Validation

After training, the system validates against 12 internal tricky cases. The following table illustrates representative boundaries and the reasoning behind each label:

Message (paraphrased)	Label	Why this classification
Congrats bro you passed the exam!	HAM	Casual congratulations — personal context, no financial hook
Free time today? Lets go out	HAM	'Free' without commercial context → personal
You wont believe what happened today lol	HAM	Conversational, slang markers
Work from home earn unlimited income daily	SPAM	No phone number, no prize → commercial scam ad
This is not a scam I promise, earn daily	SPAM	Denial + income claim → SPAM keyword hit
WINNER!! You've won £5000! Call 07123... NOW	SMISHING	Prize keyword + phone number → smishing prize
Your Barclays account locked, verify at http://...	SMISHING	Bank brand + URL → bank smishing
Free ringtone! Text NOKIA to 87021	SPAM	Shortcode + ringtone → commercial SMS
Fantasy football, win £250k dream team	SPAM	Prize WITHOUT phone → commercial, not smishing

6. Graphical User Interface

6.1 Application Flow

The application launches a tkinter window (1020×800 px, dark theme). Model initialisation runs in a background thread so the UI remains responsive.

- First run: a training log panel displays step-by-step progress while all six models train (~2 minutes). The log is replaced by the main interface once training completes.
- Subsequent runs: saved .pkl files are loaded from saved_models_final/ and the GUI opens immediately.

6.2 Interface Components

- Message input area: multi-line scrollable text box with placeholder text.
- Classify button: triggers inference on the entered text.
- Clear button: resets the input and results.

- Sample message buttons: five one-click examples (HAM, SPAM, Prize Smishing, Bank Smishing, Tricky).
- VERDICT banner: colour-coded result strip (green for HAM, red for SPAM, purple for SMISHING) with the majority vote and agreement count.
- Results table: per-model breakdown — algorithm name, predicted label, confidence %, and per-class probability breakdown (HAM / SPAM / SMISHING).
- Debug bar: lists which hand-crafted features fired for the current message, useful for understanding the prediction.

6.3 Colour Scheme

The GUI uses a consistent dark theme to visually separate the three classes:

- HAM: #1D9E75 (teal-green).
- SPAM: #E24B4A (red).
- SMISHING: #9B8EF5 (purple).
- Background: #0F1117 (near-black). Accent: #6C63FF (indigo).

7. Project Structure

File / Folder	Purpose
<code>spam_detector.py</code>	Main script — training pipeline + tkinter GUI (single file)
<code>A Balanced Dataset for.../Dataset_10191.csv</code>	10,191-row balanced dataset (HAM/SPAM/SMISHING, 3,397 each)
<code>saved_models_final/</code>	Auto-generated folder — saved model .pkl files after first run
<code>saved_models_final/tfidf.pkl</code>	Fitted TF-IDF vectorizer (6,000 n-gram features)
<code>saved_models_final/le.pkl</code>	Label encoder (ham / spam / smishing → 0/1/2)
<code>saved_models_final/scaler.pkl</code>	MaxAbsScaler for hand-crafted features
<code>saved_models_final/model_*.pkl</code>	One file per trained model (6 models total)
<code>Visualizations/</code>	Pre-generated EDA and model evaluation plots
<code>Spam_Ham_Smishing_Detection.ipynb</code>	Jupyter notebook — exploratory version of the project

7.1 Visualizations Included

- `plot_01_eda_overview.png` — dataset EDA summary (class counts, message length distributions).
- `plot_02_flag_distribution.png` — URL / EMAIL / PHONE flag frequencies per class.
- `plot_03_correlation_matrix.png` — feature correlation heatmap.
- `plot_04_wordclouds.png` — word clouds for HAM, SPAM, and SMISHING.
- `plot_05_split.png` — train/test split visualisation.
- `plot_06_model_comparison.png` — accuracy bar chart across all six models.
- `plot_07_confusion_matrices.png` — confusion matrices per model.

- plot_08_roc_curves.png — ROC curves (one-vs-rest) for each model.
- plot_09_overfitting.png — train vs. test accuracy gap analysis.
- plot_10_learning_curves.png — learning curves showing data efficiency.
- plot_11_crossval.png — k-fold cross-validation scores per model.
- plot_12_feature_importance.png — Random Forest feature importance ranking.

8. Installation & Usage

8.1 Requirements

- Python 3.11 or newer.
- scikit-learn, pandas, numpy, scipy — install with:

```
pip install scikit-learn pandas numpy scipy
```

- tkinter — included in the standard library for most Python distributions. On Ubuntu/Debian: `sudo apt install python3-tk`.

8.2 Running the Application

One command is all that is needed:

```
python spam_detector.py
```

- First run: automatically trains all models and saves them to `saved_models_final/`. Opens the GUI when complete (~2 minutes).
- Next runs: loads saved models and opens the GUI immediately.

8.3 Retraining

If predictions seem incorrect after a scikit-learn upgrade or after modifying the source code, force a full retrain by deleting the saved models folder:

```
rm -rf saved_models_final/ python spam_detector.py
```